

SMART IRRIGATION SYSTEM

FINAL REPORT

Daniel Barreiro Torres

Alejandro Pellicier

Professor: Daniel Cruz Ramírez

Due Date: May 14, 2026

Contents

Introduction	3
System Overview.....	4
Complete System Operation.....	4
Physical and Programmatic Function of the Components	4
Special Programming Libraries.....	6
External Sensors Highlighted	6
How the Components Work Together	6
Technical Deep Dive.....	8
Bluetooth module.....	8
Solenoid valve.....	9
Power Overview and Analysis	10
Code Architecture	12
Working Tinker CAD link	13
Conclusion.....	14
References.....	15

Introduction

The proposed project is a smart self-watering irrigation system designed to monitor soil conditions and regulate watering automatically for small-scale plant cultivation. The system will use an Arduino Uno R3 with a Bluetooth and ZS-040 module to read environmental data and control irrigation behavior.

The main purpose of this project is to automate plant watering while allowing the user to select different cultivation modes for plants that grow in Puerto Rico, such as beans, peppers, and tomatoes. Each crop type will have different watering parameters which will be chosen with a state machine to adjust the quantity or duration of water delivery according to the selected mode. This creates a more customized and efficient irrigation process instead of applying the same watering pattern to every plant.

The system is intended for users who want a simple low-cost automated irrigation solution for home gardening, educational demonstrations, or small indoor plant setups. In addition to autonomous operation, the user will also be able to monitor the system in real time and switch to manual control through some buttons and a menu will be displayed on a screen. Choose the plant that will be watered and choose a mode (auto or manual). A Bluetooth serial interface is not required but will try to be implemented to make it more accessible for the public through an already existing application.

System Overview

Complete System Operation

The system begins by reading the soil moisture level from the external sensor. This reading is processed by the Arduino and used to determine whether the plant requires watering. The user can select between different crop modes, such as tomato, pepper, or bean mode. Each crop mode can have its own moisture threshold or watering duration, allowing the irrigation process to be more specific instead of applying the same watering behavior to every plant.

The system operates in two major modes: automatic mode and manual mode. In automatic mode, the Arduino decides when to water the plant based on the soil moisture reading and the selected crop state. In manual mode, the user can activate watering directly through a push button or through Bluetooth serial communication. The selected mode, crop type, and sensor readings are displayed on the LCD and can also be sent to a phone through the Bluetooth module.

Physical and Programmatic Function of the Components

The Arduino UNO R3 functions as the central processing unit of the project. Physically, it connects to the sensors, communication module, display, buttons, and relay. Programmatically, it manages the main logic of the system, including reading sensor values, updating the LCD, receiving Bluetooth commands, checking the selected state, and controlling the irrigation output.

The soil moisture sensor is the main external sensor in the system. Physically, it is placed in the soil and detects the amount of moisture present by measuring the electrical behavior of the soil. Wet soil and dry soil produce different readings, allowing the Arduino to determine whether irrigation is needed. Programmatically, the Arduino reads the sensor value and compares it with the threshold assigned to the selected crop mode.

The ZS-040 Bluetooth module allows wireless communication between the Arduino and a mobile phone. Physically, it connects to Arduino through serial communication pins. Programmatically, it allows the system to send real-time information to the user, such as soil moisture level and current mode, while also receiving commands from the phone. This supports remote monitoring and manual control of the irrigation system.

The 16x2 LCD display with I2C module provides local feedback to the user. Physically, it displays text such as the selected crop, current mode, and soil moisture reading. Programmatically, the Arduino updates the display whenever important system information changes. Because the LCD uses an I2C module, it requires fewer Arduino pins than a regular parallel LCD connection.

The push buttons provide direct physical control of the system. They may be used to change modes, select a crop, or activate manual watering. Programmatically, the buttons can be handled through hardware interrupts with capacitor debounce. This allows the system to respond quickly to user input without constantly checking the button state inside the main loop.

The relay module acts as an electrical switch between the low-power Arduino control circuit and the higher-power solenoid valve circuit. Physically, it allows the Arduino to safely control the valve without powering it directly from an Arduino pin. Programmatically, the relay is activated or deactivated depending on whether the system determines that water should flow.

The solenoid valve controls the water flow. Physically, the valve opens when its internal coil is energized and closes when power is removed. Programmatically, the Arduino does not control the valve directly; instead, it controls the relay, and the relay controls power to the solenoid valve. This makes the valve the final actuator of the irrigation system.

The 12V rechargeable battery provides the main power source for the system. The LM2576 DC-DC step-down converter reduces the voltage to a level suitable for the

Arduino and other 5V components. These components are not controlled by the program, but they are necessary for providing stable power to the circuit.

Special Programming Libraries

The project may require specific Arduino libraries depending on the final implementation. The LCD display with I2C module requires an I2C LCD library, commonly used with the Arduino Wire library. The Bluetooth module can communicate through the Arduino serial interface, and a software serial library may be used if the hardware serial pins are reserved for debugging through the computer. The soil moisture sensor, relay, push buttons, and solenoid valve do not require special libraries because they can be controlled using standard Arduino input and output functions.

External Sensors Highlighted

The main external sensor in this project is the soil moisture sensor. This sensor provides the environmental data needed for automatic irrigation. It is the key input that allows the system to make decisions based on the actual condition of the soil. The proposal's block diagram also references measurements such as moisture and temperature, but the main sensor clearly described in the system input is the soil moisture sensor. If a temperature sensor is added to the final version, it should be included as an additional external sensor in the components section.

How the Components Work Together

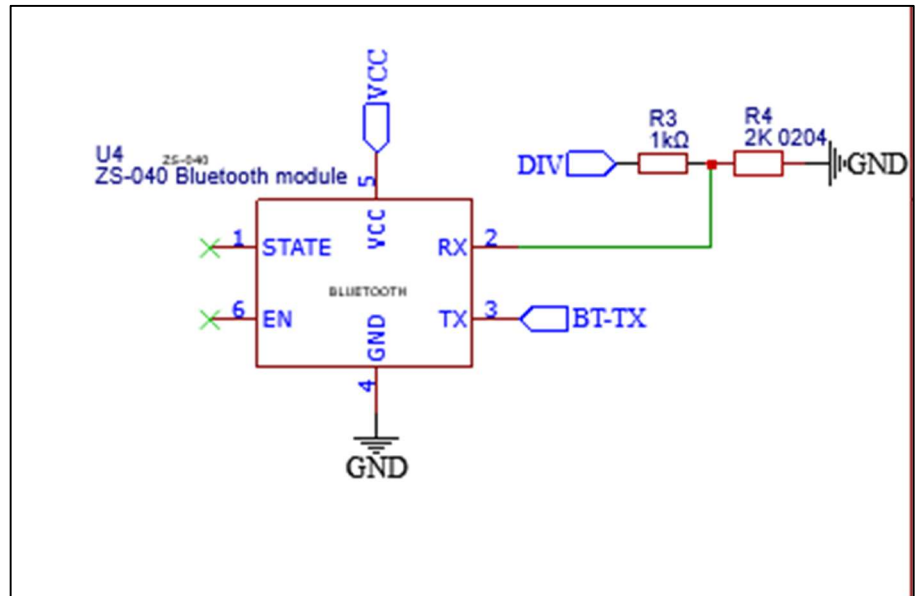
All components work together as an automated control system. The soil moisture sensor measures the condition of the soil and sends that information to the Arduino. The Arduino processes the reading and checks the current crop and operating mode. If the system is in automatic mode and the soil is below the required moisture level, the Arduino activates the relay. The relay then powers

the solenoid valve, allowing water to flow to the plant. Once watering is complete, the relay turns off and the valve closes.

At the same time, the LCD display provides local system information, while the Bluetooth module sends data to a phone and allows the user to send commands. The push buttons provide an additional local method of control, especially if Bluetooth is unavailable. This combination of automatic sensor feedback, manual override, wireless monitoring, and actuator control makes the system more flexible and reliable.

Technical Deep Dive

Bluetooth module



The Rx pin in the Bluetooth module must receive a stable 3.3v. To avoid overloading the Arduino's voltage regulator, a voltage divider is made with an output voltage of 3.3v:

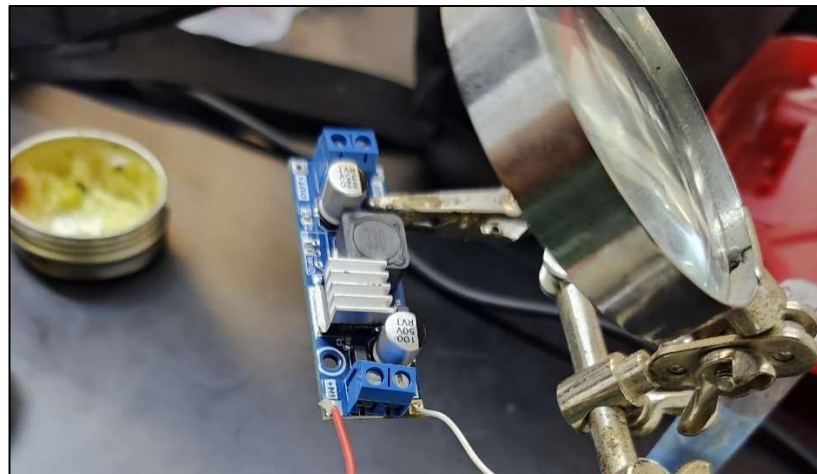
$$V_{div} = \frac{V_{in} * R_1}{R_1 + R_2} = \frac{5v * 1K\Omega}{1K\Omega + 2K\Omega} = 3.33v$$

This module uses the Arduino's Tx and Rx for Serial Communication. The downside is that while the module is connected to those pins, Serial Communication is busy and cannot communicate through USB. This caused significant pain in the troubleshooting process. A workaround we found was using a dip switch to connect and disconnect the pins as needed. This way we don't have to physically remove the wires every time we need to upload a code or communicate with a computer through USB.

¹ Bluetooth Module connection with voltage divider

Solenoid valve

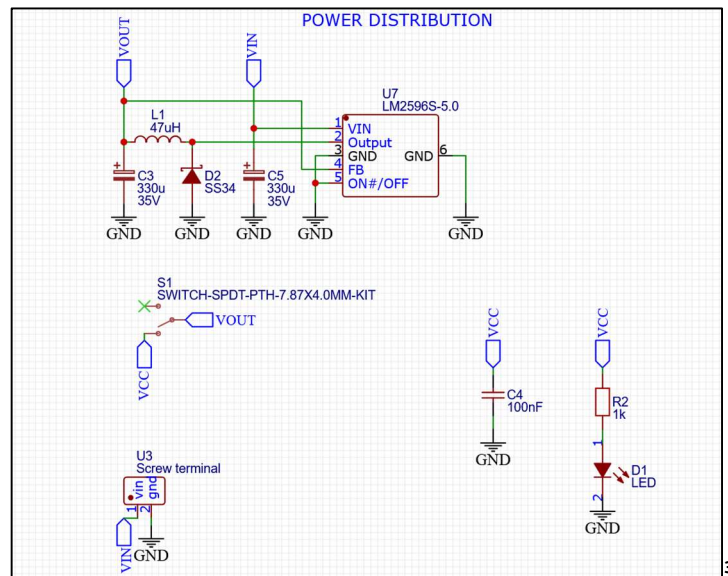
The solenoid valve which oversees controlling the flow of water requires 12 v while the Arduino needs 5v. The DC-DC Buck converter receives a input voltage of 12v which is stepped down to 5v. But a common node on the positive pin needs to be shared with the common pin in the relay module. To avoid cluttering the screw terminals, the converter has some adjacent holes where we could solder the 12v input, while using the screw terminal exclusively for the relay common terminal.



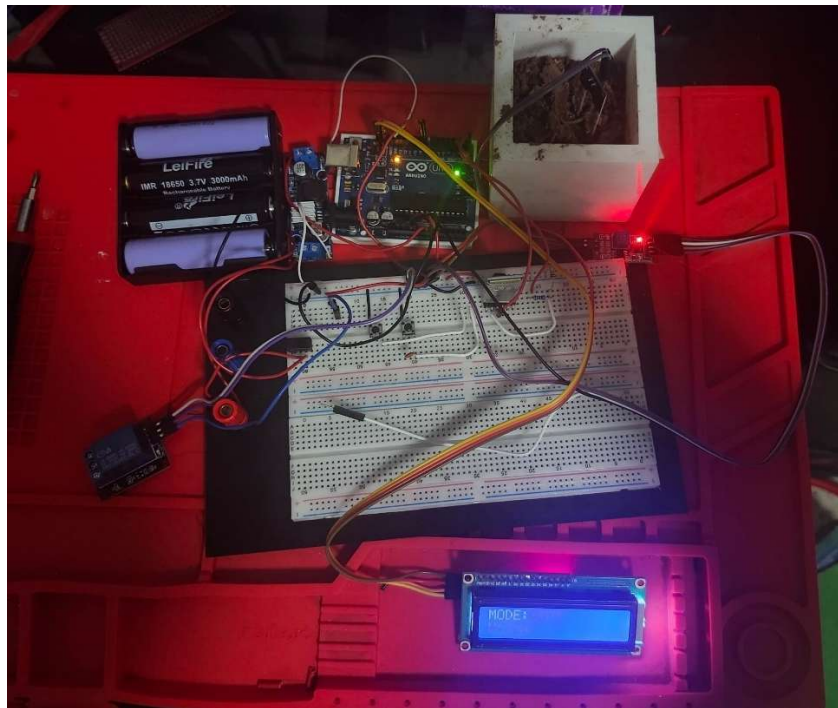
However, our solenoid valve, like most valves, require some pressure even while activated. Feeding the valve with water through gravity generated some flow. Additionally, for more flow of water the valve would require a pump. This creates a scenario where the solution is out of the scope of the project. To avoid spilling water on electronics and avoid adding a pump, a solution is to feed water by gravity, while adding a LED that activates when the solenoid signal is high. This way, the user can still visualize that the valve is turning on.

² DC-DC Buck converter with soldered wires

Power Overview and Analysis



3



4

- ³ Power distribution overview
- ⁴ Prototype measuring moisture

Inter American University of Puerto Rico – Bayamón Campus
Computer Engineering Department
ELEN 4020 – Microcontrollers

Component	Voltage	I_{max}	Power	Always on?
Arduino	5	0.05	0.25	Yes
LCD	5	0.04	0.2	Yes
Moisture sensor	5	0.02	0.1	No
Relay module	5	0.07	0.35	No
Solenoid valve	12	0.56338	6.76056	No
Bluetooth Module	5	0.028	0.14	Yes
		Total power	7.80056	

With four 3.7v 18650 batteries in series, the total voltage is 14.8v. With a rating of 3000MAH, the energy stored in watt-hours is 44.4Wh. Converters are not ideal and have some energy losses due to heat and other factors. Assuming an efficiency rating of 0.80 for our converter, our real watt-hour rating is given by:

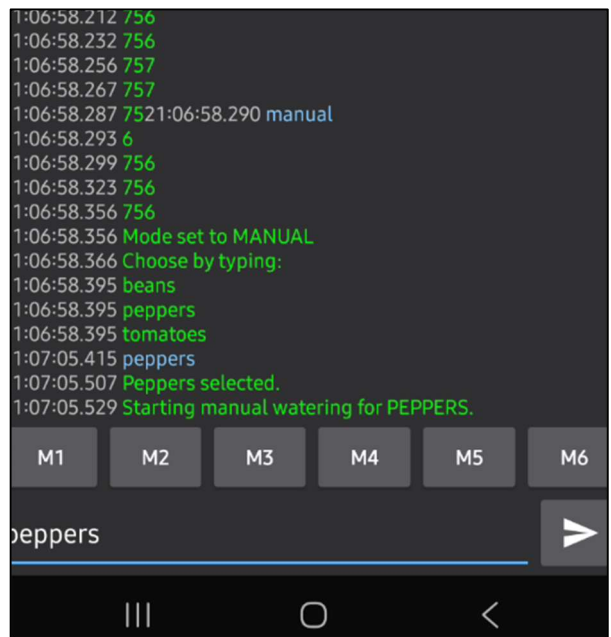
$$Wh_{real} = Wh_{Theoretica} \cdot \beta = 44.4Wh \cdot 0.80 = 35.52Wh$$

Assuming all compopnents are powered on constantly at the maximum allowable rating, We can calculate approximately how long we could power the whole system:

$$Wh_{total} = \frac{35.52Wh}{7.80059W} = 4.553519H$$

Which means we could run the whole system for 4 hours and 33 minutes.

Code Architecture



The screenshot displays a mobile application interface. The top half shows a log of system events with timestamps and data values in various colors (green, blue, red). The log includes entries for sensor readings (e.g., 756, 757, 6) and mode changes (e.g., 'Mode set to MANUAL', 'Peppers selected', 'Starting manual watering for PEPPERS'). The bottom half of the interface features a control panel with six buttons labeled M1 through M6. Below these buttons is a text input field containing the word 'peppers' and a right-pointing arrow button. The entire interface is set against a dark background.

The project's structure includes the following programming concepts as follows:

1. Finite State Machine

The crop selection and operating modes are managed through a state machine. It includes two states, or modes. These are: automatic mode and manual mode.

2. Classes

Each mode's logic is handled by classes, Auto and Manual respectively. Both contain their own methods of reading serial commands, with Auto specifically making use of the analog data obtained from the moisture sensor.

3. Conditional Logic

While in auto mode, the user can choose their crop of choice through serial commands. Each crop will have their own threshold for watering, having the system start automatically if the current moisture of the water is below that of the crop's threshold, and stop once it reaches that threshold.

⁵ Example of state machine change through mobile application

While in manual mode, the program will ignore all sensor data and focus solely on incoming serial commands. The Manual class will divide the command into two parts, the type of timer the user wants and the amount of time the user wants. For example, if the user gives the command timer.s=5, the water will be on for exactly 5 seconds, while if the user provides timer.m=5, the water will be on for five minutes. The remaining time will be displayed in the LCD screen

3. Functions / Modular Programming

The program is divided into functions such as reading sensors, updating the LCD, handling Bluetooth commands, and controlling irrigation. This will improve readability and maintenance.

4. Serial Communication

Bluetooth serial communication is used to send real-time data to the user and receive commands from a mobile device. Through serial communication, the user has complete control over the system. They will be able too pause, change modes, select the crop and a timer.

5. Analog Input Processing

The soil moisture sensor provides analog input data that is interpreted and mapped into a meaningful humidity range. This serves as the threshold for watering

6. Hardware Interrupts

Pausing and switching modes are also handled by Interrupt Service Routines.

6. Output Control / Actuation

Digital outputs are used to switch the solenoid valve on or off through a proper driver interface.

Working Tinker CAD link

<https://www.tinkercad.com/things/078oVTi38bm-final-projectmanus>

Conclusion

Being a prototype, it is normal for some implementations to not be applicable to field work. However, there were some important improvements that must be made for the next iteration or a product. Our observations are as follows:

1. Bigger LCD display - A bigger lcd display would make the menu clearer and more intuitive for the average user.
2. Casing - Putting the project in a case provides a barrier of protection against electrical shocks and water damage. It also makes it easy to install in a garden.
3. PCB - Creating a printer circuit board of our circuit board provides a more modular, smaller design that is more efficient long term
4. Charging - In the future, there could be an opportunity to implement a solar powered charging station for this system.

Regardless, this is a student effort that could create interest in green education and create opportunities for improvement of the agriculture industry.

References

ZS-040 Bluetooth Module - <https://protosupplies.com/product/hc-05-zs-040-bluetooth-module/>

DC-DC Buck converter datasheet Texas Instruments -
https://www.ti.com/lit/ds/symlink/lm5007.pdf?ts=1778745061673&ref_url=https%253A%252F%252Fwww.mouser.it%252F